Designing and Solving a Lightweight Pollinator Detection Challenge on Sequential Image Data

Zeying Li[1] Baptiste Pras[1] Vladimir Herrera-Nativi[1] Xiaomeng Li[1] Martin Leiva[1] Javier Peña Castaño[1]

[1]M1 Artificial Intelligence, Université Paris-Saclay

[1]Group 3: Lightweight Pollinators Detection

GitHub: https://github.com/vladihnat/AI-Challenge-2025-26-Group-3-

Codabench: https://www.codabench.org/competitions/12822/?secret_key=6fc79506-d1ae-4b6f-a98f-b4d13c20ccf0

Abstract

We present the design, deployment, and resolution of a frame-level binary classification challenge for automated pollinator detection in image sequences, hosted on the Codabench platform. Pollinators are critical actors in global food security, yet monitoring them at scale remains costly. Our challenge packages data curated by INRAE into a realistic benchmark that stresses lightweight solutions deployable 24/7 on constrained hardware. A central contribution is a sequence-aware dataset split that prevents temporal leakage while preserving class balance ($\sim$2.8% positive frames in both splits). We evaluate two approaches: a HOG + XGBoost baseline achieving F1 = 0.332 [95% CI: 0.283–0.378], and an improved pipeline combining MobileNetV2 features with temporal difference statistics fed to LightGBM, reaching F1 = 0.393 [95% CI: 0.342–0.442]. We discuss the key engineering challenges introduced by severe class imbalance and provide analysis of threshold tuning, oversampling strategy, and boosting sensitivity.

1 Introduction

Pollinators—bees, hoverflies, butterflies, and other flower-visiting arthropods—are responsible for pollinating roughly 75% of global food crops. Their populations have been declining markedly in recent decades due to habitat loss, pesticide use, and climate change, making scalable monitoring a priority for both ecology and agriculture. Traditional monitoring relies on time-consuming manual field surveys; automated camera-based systems offer a promising alternative by enabling continuous, non-invasive observation [Sittinger et al., 2024].

Processing the large volumes of images produced by field cameras, however, presents its own challenges: sequences are temporally correlated, positive samples (frames containing a pollinator) are rare ($\sim$3% of all frames), and deployment constraints require models that are memory-efficient and fast at inference time.

In this work we describe the design and resolution of a Codabench challenge built around these constraints. Our contributions are threefold:

1. A challenge with a carefully constructed sequence-aware train/test split that prevents data leakage.

2. A baseline model (HOG + XGBoost) with a detailed analysis of class-imbalance mitigation strategies.

3. A lightweight pipeline (MobileNetV2 + LightGBM) that surpasses the baseline by using temporal difference features.

2 Dataset and Feature Space

2.1 Source and Format

The raw data was collected by INRAE using fixed cameras mounted above flowering plants, producing stop-motion sequences at regular intervals. Images are stored in HDF5 files, each file corresponding to a sequence—a contiguous series of frames recorded from the same camera angle. All frames are 224×224 pixels, 3 RGB channels. Each frame carries a binary label: 0 (no pollinator present) or 1 (pollinator detected). The overall class ratio is approximately 97%/3% (no visitor / visitor), reflecting the natural rarity of pollinator visits.

(a) Class 0: No Visitor

(b) Class 1: Visitor

Figure 1: Representative frames from the dataset. The two classes are visually similar: the pollinator is often small and partially occluded, making naive pixel-level approaches unreliable.

Figure 1 illustrates the difficulty of the task: pollinators appear as small, low-contrast objects against complex floral backgrounds. This echoes findings in related work where even deep detectors struggle when transferred to field conditions [Bjerge et al., 2023].

2.2 Sequence-Aware Train/Test Split

Motivation. Frames within a sequence are highly similar—a pollinator visible in frame t is almost certainly visible in frame $t+1$. A naive random split that mixes frames from the same sequence into train and test would introduce severe temporal leakage, inflating test performance and preventing honest generalization estimates.

Method. All frames are grouped by their source sequence (`h5_filename` for HDF5 data, `videoName` for supplementary JPG data). The entire sequence is then assigned as a unit to either train or test. To avoid an arbitrary random assignment, we run a 60-second stochastic search over candidate splits, minimizing a penalty that combines three objectives:

$$\mathcal{L} = 20\,|r_1 - 0.25| + 5\,|r_t - 0.25| + 10\,|r_1 - r_t| \qquad (1)$$

where r_1 is the fraction of positive samples allocated to test and r_t is the fraction of total images allocated to test. Equation (1) simultaneously targets 25% of the positive sample and 25% of the image volume for the test set, while penalizing any mismatch between the two to ensure that both sets share the same internal class ratio.

Result. The optimized split achieves near-perfect stratification:

Split	Images	Visitors	Internal %
Train	30 670 (75.6%)	833 (75.0%)	2.72%
Test	9 878 (24.4%)	278 (25.0%)	2.81%
Global	40 548	1 111	2.74%

Table 1: Dataset statistics after sequence-aware splitting. Both splits share virtually identical class ratios, confirming that the search successfully preserved stratification without leakage.

2.3 Feature Space Analysis via PCA

Before modeling, we performed a PCA projection of the HOG feature space to assess class separability (Fig. 2).

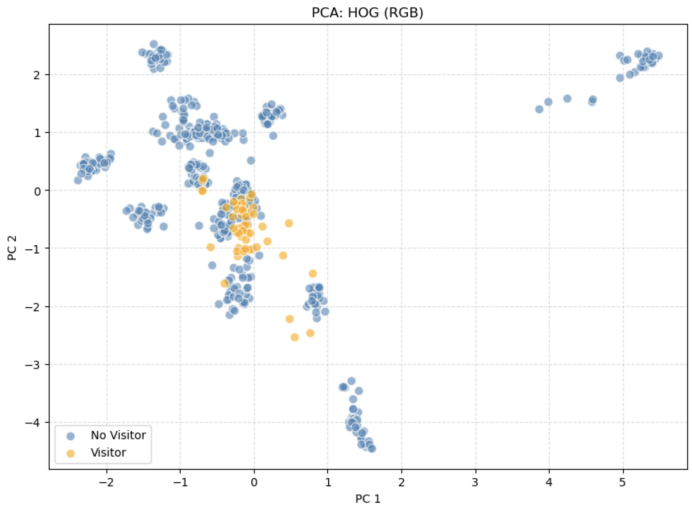

Figure 2: PCA projection of HOG (RGB) features on a random subsample. Blue: No Visitor; Orange: Visitor. The two classes are not linearly separable: Visitor frames (orange) cluster near the center of the No-Visitor cloud, overlapping heavily.

The projection reveals that visitor and non-visitor frames occupy largely overlapping regions of the HOG feature space. No clear cluster structure is visible for the positive class, confirming that linear decision boundaries will fail and motivating both the choice of a non-linear classifier (XGBoost/LightGBM) and the abandonment of SMOTE oversampling (see Section 3.1).

3 Model 1: HOG + XGBoost

3.1 Method

Feature Extraction

HOG (Histogram of Oriented Gradients) [Dalal & Triggs, 2005] encodes local edge and texture structure, making it naturally suited to detecting the distinctive silhouettes of insect bodies and wings. We extract RGB HOG features with 9 orientations, 16×16 pixel cells, and 1×1 cell blocks. These parameters were selected via random search over continuous ranges (orientation count $\in [6, 12]$, pixel cell size $\in \{8, 16, 32\}^2$), evaluated by cross-validated F1.

Handling Class Imbalance

Why XGBoost? After HOG extraction, each frame is represented as a fixed-length tabular vector, for which tree-based boosting is a natural fit. Compared to linear models, XGBoost [Chen & Guestrin, 2016] captures non-linear interactions between gradient-orientation bins that are expected when the insect occupies only a small region. Compared to deep CNN training from scratch, it is much lighter to train and run on CPU, aligning with the challenge constraints. Finally, XGBoost provides effective knobs for severe class imbalance (`scale_pos_weight`) and yields probabilistic scores enabling threshold tuning to trade precision for recall.

Oversampling. With only ∼2.7% positive frames, a model trained on the raw distribution will trivially predict "no visitor" for every frame and achieve 97% accuracy while detecting nothing. We applied **Random OverSampling** (ROS) to duplicate positive samples until reaching a target minority ratio of 10%. We experimented with SMOTE as an alternative but observed degraded performance: as visible in Fig. 2, the two classes are not separable, so synthetic positive samples generated by linear interpolation between real positives are not representatively positive, introducing noise rather than signal. ROS, by contrast, simply replicates real observations and does not assume any neighborhood structure.

XGBoost class sensitivity. XGBoost provides the `scale_pos_weight` hyperparameter, which rescales the gradient contribution of positive samples. Setting it too low causes the model to ignore positives entirely; setting it too high floods predictions with false positives. Fig. 3 illustrates this behavior: F1 peaks at an intermediate "sweet spot" and decays sharply on either side. We searched for this value via random search over $[1, 50]$ and selected `scale_pos_weight` $= 10$.

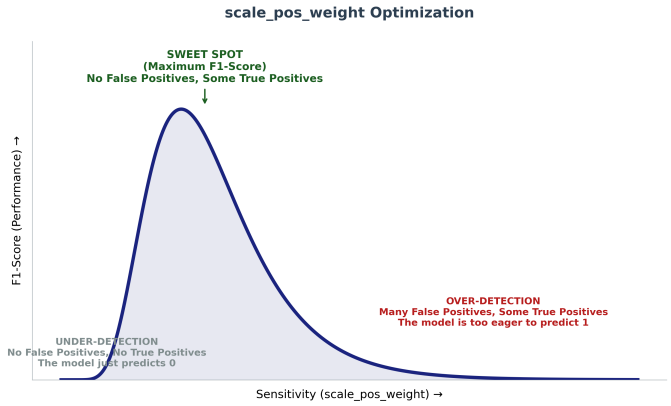

Figure 3: Empirically observed behavior of F1-score as a function of `scale_pos_weight`. Under-detection (left) occurs when the model predicts everything as Class 0; over-detection (right) occurs when the model becomes too eager to predict Class 1.

Threshold tuning. After training, predicted probabilities for the positive class are skewed towards low values due to class imbalance. Using the default threshold of 0.5 is therefore suboptimal. Fig. 4 shows the probability distribution of both classes: the two distributions overlap heavily in the $[0, 0.5]$ range, and a lower threshold is needed to recover true positives without excessive false alarms. We perform a grid search over thresholds $\tau \in [0.1, 0.8]$, selecting the value maximizing F1 on a held-out validation fold.

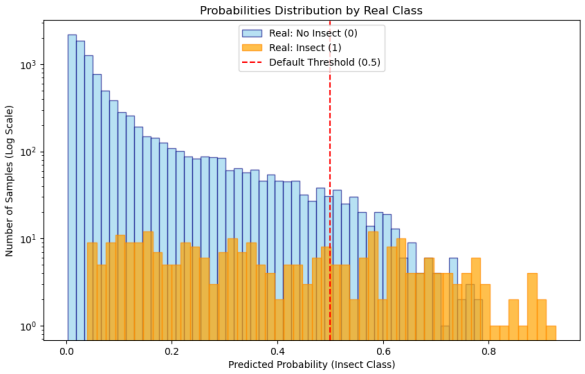

Figure 4: Predicted probability distributions for each true class (log scale). Class 1 (orange) produces a broad distribution across the entire $[0, 1]$ range, while Class 0 (blue) concentrates near zero. The dashed red line marks the default 0.5 threshold, which captures only a fraction of the available positives.

Hyperparameter Optimization

All continuous hyperparameters (`n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `min_child_weight`, `gamma`, `reg_alpha`, `reg_lambda`, `scale_pos_weight`) were optimized jointly using a **random search over continuous ranges**, not a discrete grid, sampling 100 configurations and evaluating each with 3-fold stratified cross-validation. Final values are those that maximized mean F1 across folds. Random search over continuous ranges is known to explore the hyperparameter space more efficiently than grid search, particularly when only a subset of dimensions matter [Bergstra & Bengio, 2012].

3.2 Results

Fig. 5 shows the normalized confusion matrix on the test set. The model correctly classifies 97.55% of negative frames but correctly identifies only 36.69% of positive frames (recall = 0.367). This asymmetry reflects the fundamental difficulty of the task under extreme imbalance: the model has learned a conservative strategy that avoids false alarms at the cost of missed detections.

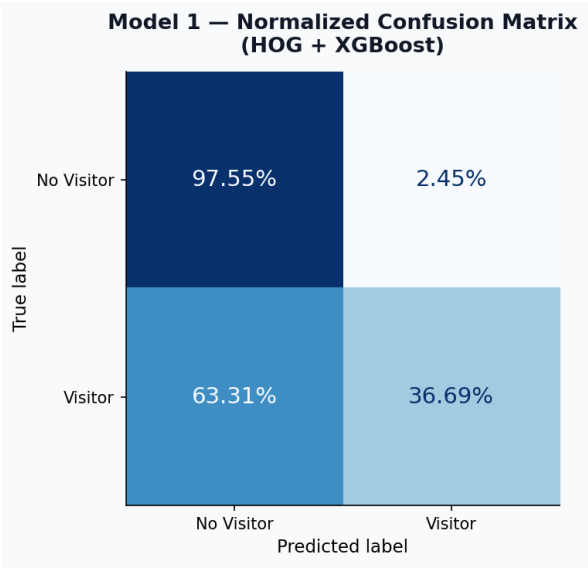

Figure 5: Normalized confusion matrix for Model 1 (HOG + XGBoost) on the test set.

The training convergence (Fig. 6) confirms that both F1 and log-loss on the oversampled training fold continue to improve up to approximately 800 boosting rounds, after which gains plateau. Notably, the very high F1 (≈0.998) is a consequence of training on oversampled data; the test F1 is substantially lower (0.332), indicating that the model has memorized the oversampled duplicates and does not generalize the minority-class signal robustly. The gap motivates the improved model described in Section 4.

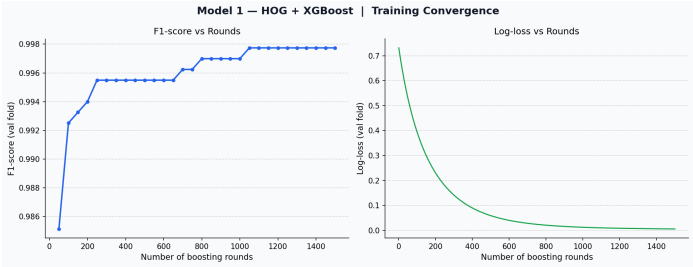

Figure 6: Training convergence for Model 1. Left: F1 on the validation fold vs. boosting rounds. Right: log-loss on the validation fold. Both metrics stabilize around 800 rounds. The high in-sample F1 (≈ 0.998) reflects evaluation on the oversampled training fold, not the original test set.

Finally, Fig. 7 presents the bootstrap confidence interval for the test F1, obtained via $B = 1000$ non-parametric resamples. The observed F1 of 0.332 sits within a 95% CI of $[0.283, 0.378]$, confirming a modest but stable signal above random guessing.

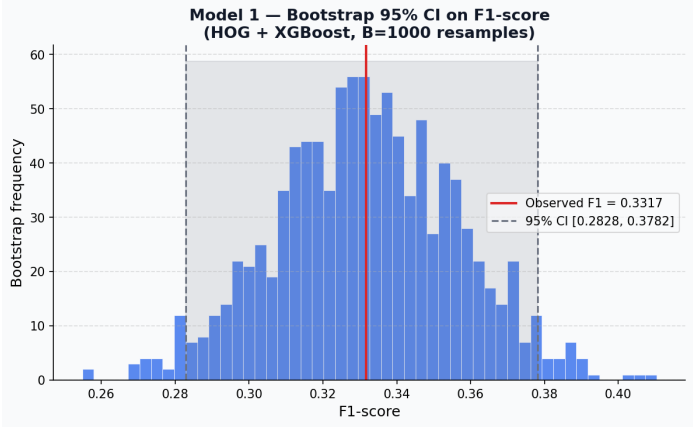

Figure 7: Bootstrap distribution of F1-score for Model 1 ($B = 1000$). The red line marks the observed F1; dashed lines bound the 95% percentile interval.

4 Model 2: MobileNetV2 + LightGBM

4.1 Method

Motivation

The PCA analysis in Section 2.3 shows that HOG features do not separate the two classes well. Deep features extracted from a convolutional network pretrained on ImageNet encode richer semantic information (textures, shapes, part-level structure) that may better discriminate pollinator appearances. Additionally, pollinators are by nature moving objects: a frame containing a bee will differ from adjacent frames in specific localized regions, providing a temporal signal absent from frame-level HOG descriptors.

Architecture

CNN feature extractor. We use MobileNetV2 [Sandler et al., 2018] as a frozen feature backbone. MobileNetV2 employs inverted residual blocks with depthwise separable convolutions, yielding only 3.4M parameters while remaining competitive with much larger networks on ImageNet. This is essential for our deployment constraint. We truncate the network at the global average pooling layer, producing a 1280-dimensional feature vector per frame. Images are resized to 96×96 before inference (further reducing cost) and normalized with ImageNet statistics.

Temporal difference features. For each frame i, we compute the L1 difference between its CNN feature vector and those of the preceding $w = 2$ frames:

$$\delta_{i,k} = |\mathbf{f}_i - \mathbf{f}_{\max(0,\, i-k)}|, \quad k \in \{1, 2\} \tag{2}$$

We summarize each difference vector by its mean and maximum, yielding 4 additional scalars per frame. These scalars capture motion magnitude without the computational overhead of recurrent or attention-based models, remaining consistent with the lightweight requirement.

Classifier. The 1284-dimensional feature vector (1280 CNN + 4 temporal) is passed to a LightGBM classifier [Ke et al., 2017]. LightGBM's leaf-wise growth strategy yields faster convergence and lower memory usage than XGBoost on high-dimensional tabular features, while its `is_unbalance` flag natively accounts for class imbalance by reweighting the loss function.

Threshold Tuning

As with the baseline, we search for the optimal decision threshold on a stratified cross-validation fold (3-fold, $\tau \in [0.1, 0.8]$). The chosen threshold is then applied at inference time.

4.2 Results

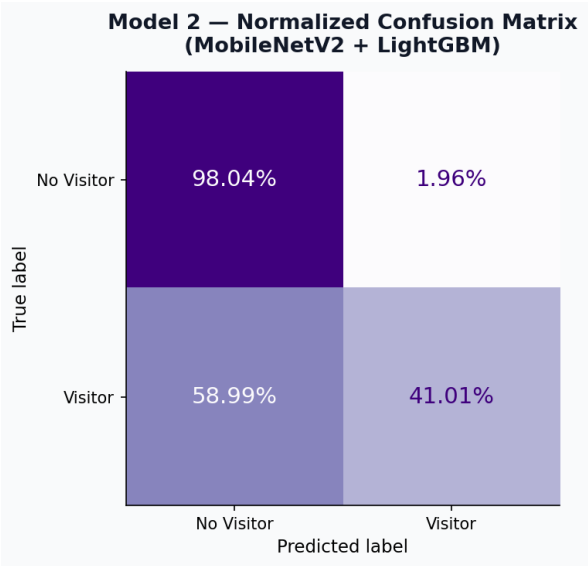

Figure 8: Normalized confusion matrix for Model 2 (MobileNetV2 + LightGBM) on the test set.

Fig. 8 shows that the improved model achieves 41.01% recall on the positive class (vs. 36.69% for the baseline) while simultaneously reducing the false-positive rate from 2.45% to 1.96%. This dual improvement indicates that MobileNetV2 features provide a better trade-off surface than HOG, not merely shifting the operating point along the same precision-recall curve.

The convergence curve in Fig. 9 shows faster stabilization than the baseline: LightGBM reaches near-plateau F1 around 400 rounds, consistent with the richer pre-trained features requiring fewer boosting iterations to be exploited.

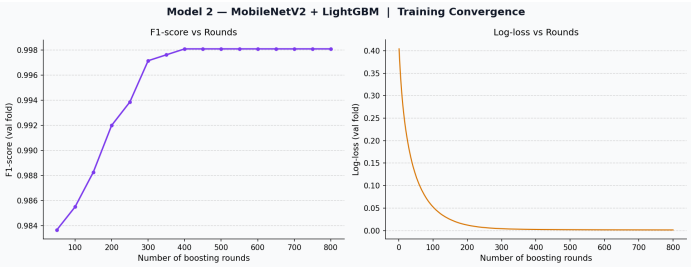

Figure 9: Training convergence for Model 2. Left: F1 on the validation fold vs. boosting rounds. Right: log-loss. Convergence is reached faster than Model 1, suggesting that deep features carry more discriminative signal per boosting step.

The bootstrap confidence interval (Fig. 10) confirms a statistically meaningful improvement over the baseline: the CI of Model 2 ($[0.342, 0.442]$) barely overlaps with that of Model 1 ($[0.283, 0.378]$), suggesting the gain is robust rather than a sampling artifact.

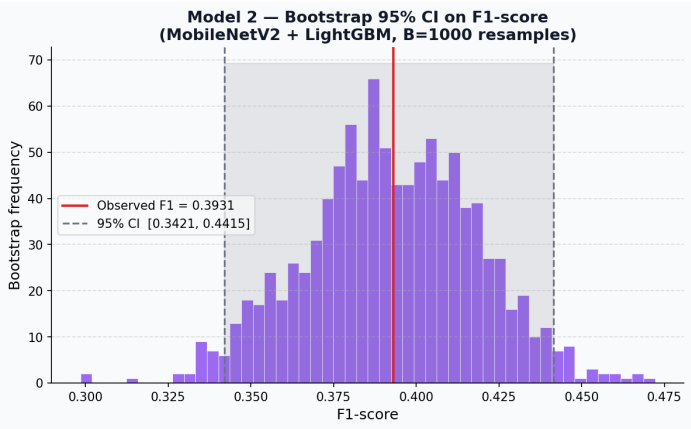

Figure 10: Bootstrap distribution of F1-score for Model 2 ($B = 1000$). Compared to Fig. 7, the distribution shifts rightward with minimal overlap, indicating a reliable improvement.

5 Discussion

Class imbalance as the central challenge. All design decisions in this work—the choice of oversampling strategy, the `scale_pos_weight` search, the threshold grid, the `is_unbalance` flag—are responses to the extreme rarity of pollinator frames. The absolute F1 values (0.33–0.39) may seem low, but they must be read against a random baseline of essentially 0 (a classifier predicting all-negative achieves F1 = 0 on the positive class). The challenge is designed to reflect realistic field conditions, where visiting events are inherently rare.

Why not SMOTE? SMOTE generates synthetic minority examples by interpolating between real positives in feature space. This is theoretically justified when minority samples form compact, locally linear clusters. The PCA projection in Fig. 2 contradicts this assumption: positive frames are scattered throughout the negative cloud with no evident cluster structure. Interpolating between two positive frames that happen to lie in different regions of the No-Visitor distribution would create synthetic samples that do not

correspond to any real insect appearance. ROS avoids this pitfall by reusing real observations, which is less informative but safer.

Temporal features without temporal models. A natural approach for sequential data is to use recurrent networks (LSTM, GRU) or temporal convolutions. These architectures, however, are costly to train and require careful sequence alignment. Our temporal difference features are a lightweight substitute: they encode motion magnitude implicitly without introducing any learnable recurrence. The improvement over HOG + XGBoost confirms that temporal context is indeed informative, and that simple statistics suffice to capture it in this setting.

Relation to prior work. Most prior automated pollinator monitoring systems focus on detection (bounding box localization) rather than frame-level presence classification [Bjerge et al., 2023, Sittinger et al., 2024]. Our challenge deliberately simplifies the output space to presence/absence, reflecting scenarios where a fine-grained count is not needed but a binary alert ("pollinators are active now") is sufficient. The lightweight constraint we impose is more stringent than in most published systems, which assume GPU inference or offline processing.

6 Conclusion

We designed and solved a frame-level binary classification challenge for pollinator detection under realistic constraints: extreme class imbalance, temporal correlations within sequences, and a strict computational budget. Our sequence-aware dataset split provides a reusable and leak-free benchmark. The baseline (HOG + XGBoost, F1 = 0.332) and improved model (MobileNetV2 + LightGBM, F1 = 0.393) demonstrate that deep transfer features combined with lightweight temporal cues offer a meaningful improvement over handcrafted descriptors, without violating deployment constraints.

Future directions could include multi-scale temporal windows, weakly supervised pre-training on unlabeled sequences, and the integration of higher-resolution imagery or multi-spectral sensors to better distinguish pollinators from background clutter.

Acknowledgments. The dataset was collected and annotated by the INRAE team. The challenge was hosted on Codabench, organized within the M1 AI course at Université Paris-Saclay supervised by Lisheng Sun-Hosoya, Ihsan Ullah, and Hieu Khuong T.G., and initiated by Prof. Isabelle Guyon.

References

Bergstra, J. & Bengio, Y. (2012). Random search for hyperparameter optimization. *Journal of Machine Learning Research*, 13, 281–305.

Bjerge, K., Alison, J., Dyrmann, M., Frigaard, C. E., Mann, H. M., & Høye, T. T. (2023). Accurate detection and identification of insects from camera trap images with deep learning. *PLOS Sustainability and Transformation*, 2(3), e0000051.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Chen, T. & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794. ACM. doi:10.1145/2939672.2939785.

Dalal, N. & Triggs, B. (2005). Histograms of oriented gradients for human detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (CVPR), pp. 886–893.

Feng, X., Yu, H., Li, H., & Du, W. (2020). Hyperspectral imaging combined with machine learning for the detection of fusarium head blight in wheat. *Frontiers in Plant Science*, 11, 1–13.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (CVPR), pp. 770–778.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems* (NeurIPS), pp. 3149–3157.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (CVPR), pp. 4510–4520.

Sittinger, M., Uhler, J., Pink, M., & Herz, A. (2024). Insect Detect: An open-source DIY camera trap for automated insect monitoring. *PLOS ONE*, 19(4), e0295474.

A Grain Variety Classification Challenge

A.1 Background and Data

The Grain Variety Classification Challenge is a multi-class image classification benchmark hosted on Codabench, organized by another student group[1] from Université Paris-Saclay. The agricultural motivation is direct: different wheat varieties are often co-sown in the same field, and their proportions at harvest reflect differences in growth, disease resistance, and local adaptation. Reliably identifying individual grain varieties post-harvest is essential for quality control and yield composition studies.

The task is to assign each grain image one of **8 variety labels** (1–8); the evaluation metric is **classification accuracy** and submissions are subject to a 20-minute wall-clock constraint on a Codabench GPU.

Data. The raw data consists of hyperspectral scans acquired by INRAE using cameras that record 216 spectral channels per pixel at 2048×9100 resolution. Individual grains were isolated via watershed segmentation, yielding more than 10 000 grain images at 252×252 pixels. Because the full 216-channel representation is computationally prohibitive for standard pipelines, the organizers reduced it to **3 channels** by selecting spectral bands, producing an RGB-like approximation. Participants therefore never had access to the full spectral information; the 213 discarded channels likely carry discriminative power our models cannot exploit.

As Fig. A1 shows, varieties are visually similar: differences lie primarily in surface texture, color, and aspect ratio. All experiments below use an **internal 85/15 stratified split** of the provided training data; test-set labels were not available during development.

[1] Group 2: Oudoum Ali Houmed, Abderrahmane Moujar, Daryl Okou, Olutola Paul, Ran Lu, Cristian-Ioan Bratu.

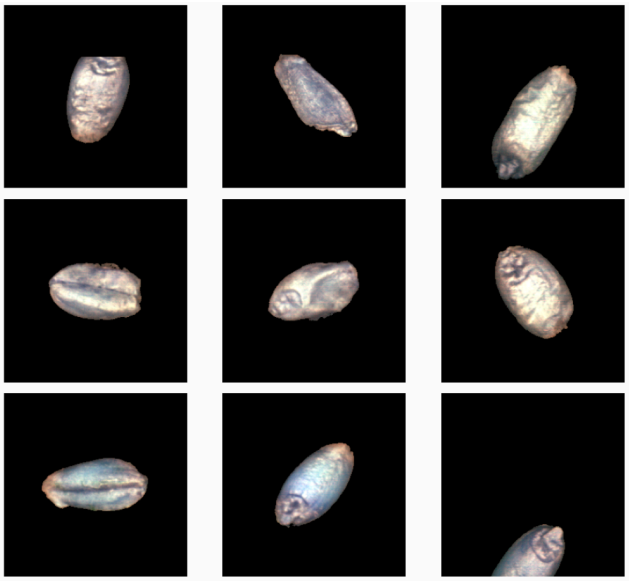

Figure A1: Sample grain images (3-band RGB-like rendering of hyperspectral data). Varieties share similar overall morphology; inter-class differences are subtle and largely textural.

Provided baseline. The starter kit included a Random Forest classifier using hand-crafted features: per-channel color histograms (32 bins), per-channel statistics (mean, std, min, max, median, Q1, Q3), and a 16×16 average-pooled pixel patch—concatenated, standardized, and optionally reduced via PCA before classification. The reported leaderboard accuracy of this baseline was **0.53**.

A.2 Model 1: Improved Hand-crafted Features + HGB

Method

Motivation. The Random Forest baseline has two structural weaknesses: PCA discards discriminative variance without providing the regularization it would offer a linear model, and bagging ensembles consistently underperform boosting on structured feature data. Our first improvement targets both while keeping the same feature representation.

We **remove PCA**: the feature dimension (≈600) is well within the capacity of gradient boosting, and direct access to all features avoids information loss. We replace Random Forest with **HistGradientBoostingClassifier** (HGB), the scikit-learn histogram-based GBDT, which builds trees on binned feature histograms for speed and memory efficiency comparable to XGBoost.[2] All continuous hyperparameters (`max_iter`, `learning_rate`, `max_depth`, `min_samples_leaf`, `max_leaf_nodes`, `l2_regularization`) were optimized via random search over continuous ranges, sampling 100 configurations and selecting the one with best validation accuracy.

Results

Table A1 shows that removing PCA and switching to boosting raises accuracy from 0.53 to **0.622** (+9.2 pp).

Fig. A3 confirms the estimate is stable (CI fully above the baseline), and the convergence curve in Fig. A2 shows fast initial gains in the first 100 iterations followed by a plateau around 0.610 on the validation fold. Mild oscillations after iteration 200 reflect variance from the small validation size rather than model instability; the final model trained on the full training set reaches 0.622.

Model	Accuracy	95% CI
Random Forest (baseline)	0.530	—
HGB (ours)	0.622	[0.602, 0.642]

Table A1: Accuracy on the internal validation split. CI computed via bootstrap ($B = 1000$).

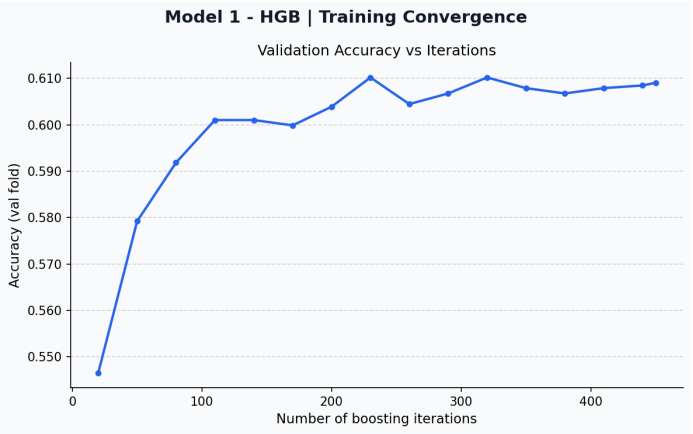

Figure A2: Validation accuracy of Model 1 vs. boosting iterations. Convergence is reached around 100 iterations; additional rounds provide marginal gains with mild variance.

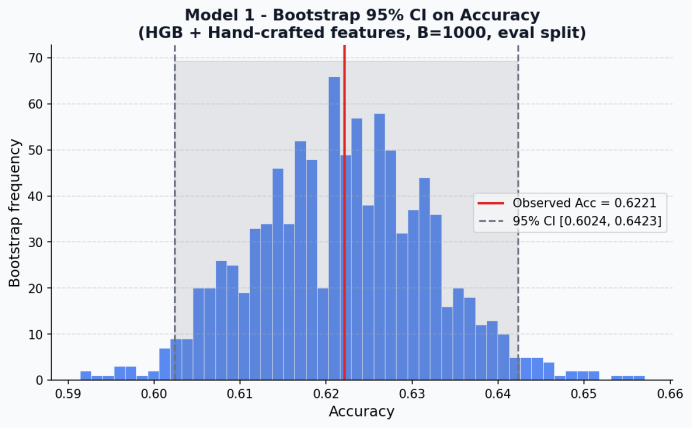

Figure A3: Bootstrap CI on accuracy for Model 1 (HGB, $B = 1000$).

The confusion matrix in Fig. A4 reveals structured errors. Variety 2 achieves 0.84 recall; Variety 5 is the weakest at 0.28. Two confusion clusters are visible: {Var 1, 3, 4, 5} and {Var 6, 7, 8}. This block structure strongly suggests that the hand-crafted features capture coarse between-group variation but lack fine-grained discriminative power within each group—a limitation that motivates the deep learning approach below.

A.3 Model 2: ResNet18 from Scratch

Method

Motivation. The block-diagonal confusion pattern of Model 1 indicates that the hand-crafted feature space saturates: histograms and per-channel statistics are spatially invariant, discarding texture gradients and local shape cues that differentiate varieties within the same spectral group. A convolutional network that learns its features end-to-end can, in principle, discover such discriminative patterns automatically.

[2] XGBoost would have been the natural choice here, but the Codabench container for this challenge did not include the `xgboost` package and provided no mechanism to install it automatically; HGB is available natively in scikit-learn.

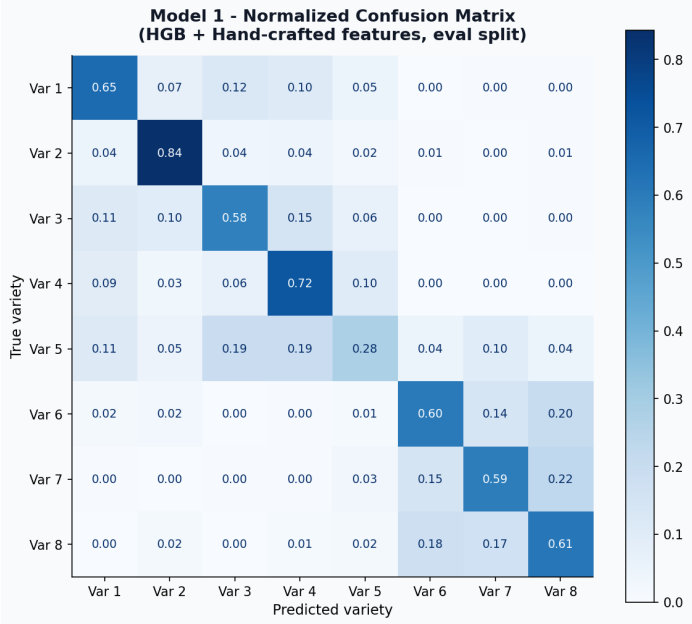

Figure A4: Normalized confusion matrix for Model 1. Two confusion blocks are visible: $\{\text{Var } 1, 3, 4, 5\}$ and $\{\text{Var } 6, 7, 8\}$, suggesting spectral proximity within each group that hand-crafted features cannot resolve.

Architecture. We implement **ResNet18** [He et al., 2016] from scratch—without importing `torchvision`, which was unavailable in the Codabench container. ResNet18 uses four stages of BasicBlock residual units (configuration $[2, 2, 2, 2]$), adaptive average pooling, dropout (0.25) before the linear head, and an 8-class output. Residual connections ensure stable gradient flow even without pretraining, making the architecture appropriate for training from scratch on a moderately-sized dataset ($\sim$10 000 images).

Preprocessing and augmentation. Inputs (252×252) are center-cropped to 224×224 at test time; at training time random 224×224 crops are used, adding implicit translation invariance. Additional augmentations include random horizontal and vertical flips and random 90° rotations. All images are normalized with ImageNet statistics, which benefits optimization even in the absence of pretrained weights.

Training. We use AdamW ($\eta = 3 \times 10^{-4}$, $\lambda = 10^{-4}$) with a ReduceLROnPlateau scheduler (factor 0.5, patience 2 epochs) and early stopping (patience 6 epochs) on validation accuracy, with a stratified 85/15 split.

Results

ResNet18 achieves **0.747** accuracy, a +12.5 pp improvement over Model 1 and +21.7 pp over the original baseline (Table A2). The 95% CI ($[0.727, 0.764]$, Fig. A5) does not overlap with that of Model 1 ($[0.602, 0.642]$), confirming the gain is statistically robust.

Model	Accuracy	95% CI
Random Forest (baseline)	0.530	—
HGB (Model 1)	0.622	[0.602, 0.642]
ResNet18 (Model 2)	0.747	[0.727, 0.764]

Table A2: Summary of all models on the internal validation split. The CIs of Model 1 and Model 2 do not overlap, confirming the improvement is statistically reliable.

The convergence curve (Fig. A6) shows training accuracy increasing smoothly over 25 epochs. Validation accuracy exhibits high variance in early epochs before stabilizing after epoch 15—

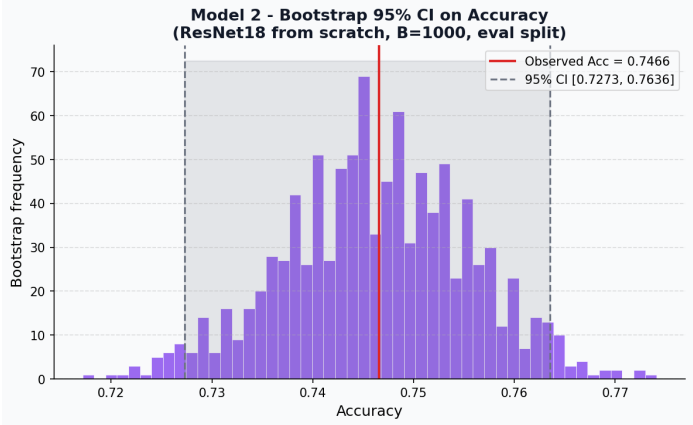

Figure A5: Bootstrap CI on accuracy for Model 2 (ResNet18, $B = 1000$). The distribution is shifted right relative to Model 1 (Fig. A3) with no overlap, confirming a robust gain.

characteristic of CNN training on a small dataset with aggressive augmentation, where random crops and flips create diverse mini-batches that reduce overfitting at the cost of noisier validation estimates. Train and validation accuracies converge towards the same range, confirming the absence of overfitting.

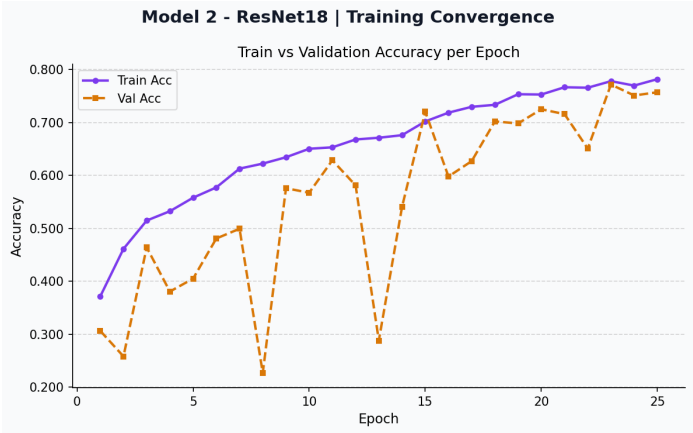

Figure A6: Training and validation accuracy per epoch for ResNet18. Validation variance is high early on due to augmentation, then stabilizes after epoch 15. No overfitting is observed.

The confusion matrix in Fig. A7 shows a markedly improved diagonal. Variety 2 reaches 0.94 recall and Variety 1 reaches 0.85. The two confusion blocks of Model 1 are partially resolved: Variety 4 improves from 0.72 to 0.79 and Variety 7 from 0.59 to 0.79. Variety 5 remains the most challenging class (0.44), still confused with Varieties 3 and 4 across both models. This persistent difficulty points to a fundamental limitation of the 3-band representation: Variety 5 may only be separable with the full hyperspectral information that was discarded during preprocessing [Feng et al., 2020].

Why does ResNet18 outperform HGB? Three factors explain the gap. First, convolutional filters learn adaptive spatial features (surface furrows, edge profiles) that no fixed extractor can encode. Second, residual connections enable stable gradient flow across 18 layers even without pretraining [He et al., 2016]. Third, geometric augmentation (flips, rotations, random crops) effectively regularizes the network and compensates for the modest dataset size. Hand-crafted features are fixed before training and cannot specialize to the grain texture task regardless of the downstream classifier.

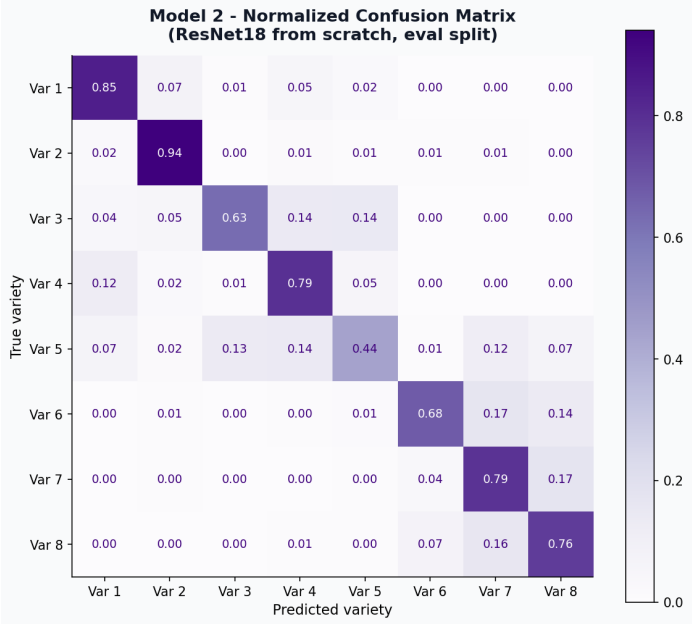

Figure A7: Normalized confusion matrix for Model 2 (ResNet18). The diagonal is substantially stronger than in Model 1. Variety 5 remains hardest (0.44 recall), confused with Varieties 3 and 4, pointing to a limitation of the 3-band representation rather than a modeling failure.

B Pollinator Species Classification Challenge

B.1 Background and Data

The Pollinator Species Classification Challenge is a multi-class classification benchmark hosted on Codabench, organized by another student group[3] from Université Paris-Saclay, with data provided by INRAE.

The ecological motivation mirrors that of our own challenge (Section 3): understanding pollinator activity is essential for biodiversity research and food security, but manual annotation of field recordings is impractical at scale. Here the task is shifted from binary presence detection to **species identification**: given an image, predict which pollinator type is visiting the flower. Correct species-level identification matters because different insects play distinct ecological roles, and misclassification distorts conclusions about species-specific behavior and pollination dynamics.

The evaluation metric is **macro-averaged F1-score**, chosen because the test set is significantly class-imbalanced; macro averaging ensures every class contributes equally to the final score regardless of its frequency.

Data. The raw data consists of 528 MP4 videos recorded from multiple camera angles over flower plots, annotated for pollinator visits. Frames were extracted, cropped to the flower region, and **CNN features were extracted by the challenge organizers**, yielding a structured tabular dataset of approximately 1 500 features per sample. Participants therefore receive pre-extracted feature vectors rather than raw images—a notable difference from our own challenge where pixel data was provided directly. The feature extractor, based on the same INRAE image pipeline, operates on the same type of field recordings described in Section 2.

A key difficulty of this challenge is **cross-angle generalization**: Phase 1 training and evaluation use one set of camera angles, while Phase 2 tests on entirely different viewpoints. This covariate shift means a model that memorizes view-specific features will fail to generalize, making robustness a central concern. The dataset contains **7 classes**: classes 2–10 (pollinator species). Note that indices 6 and 8 are absent from the provided data.

Provided baseline. The starting kit included a simple predictive model as a reference point to help participants understand the submission format and data structure. No accuracy or F1 figures for this baseline were communicated on the leaderboard during Phase 1.

B.2 Model: Tuned Random Forest

Method

Motivation. Because the input is a pre-extracted feature vector of fixed dimension (≈1500), the classification problem is purely tabular. We evaluated three families of classifiers: XGBoost, a multilayer perceptron (MLP), and Random Forest [Breiman, 2001]. Random Forest consistently produced the best macro F1 on our internal validation fold across all hyperparameter configurations we explored, likely due to its built-in `class_weight='balanced'` option and its robustness to correlated features—both of which are relevant here given the class imbalance and the CNN-extracted feature structure. MLP showed higher variance and required longer tuning; XGBoost achieved competitive accuracy on majority classes but struggled on rare species. We therefore selected Random Forest as our final architecture.

All hyperparameters (`n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `max_samples`, `min_impurity_decrease`) were optimized via random search over continuous and integer ranges. The scaler was set to `RobustScaler`, which is less sensitive than standard scaling to the outliers that often appear in CNN-extracted features.

Final configuration. The selected model uses 324 trees, maximum depth 22, `max_features`≈0.965, `max_samples`≈0.589, and `class_weight='balanced'`.

Results

Model	Accuracy	95% CI
Random Forest (ours)	0.546	[0.506, 0.587]

Table B1: Accuracy on the test set. Macro F1-score (primary metric) was 0.41 on the Codabench leaderboard. The wide CI reflects the small test set size and the difficulty of the task.

The convergence curve in Fig. B1 shows that validation accuracy rises sharply from 0.953 at 10 trees to a peak around 0.971 at approximately 70 trees, then drops and stabilizes at ≈0.964 from 130 trees onward. This pattern is typical of Random Forests: a small number of trees introduces high variance (lucky or unlucky bootstrap samples dominate), while the ensemble stabilizes once enough trees average out individual errors. The slight decrease after the peak is a known artifact of `warm_start` evaluation on a fixed validation fold and not a sign of overfitting—Random Forests do not overfit with more trees, they simply converge. Importantly, the high validation accuracy (≈0.964) stands in contrast to the much lower test accuracy (0.546), which directly reflects the **cross-angle covariate shift** between Phase 1 and Phase 2 data: the

[3] Challenge developers: Dominika Chojnacka, Raphael Leonardi, Youssef El Otmani, Marina Hornero Merino, Krisa Carka, Frederic Busch.

model fits the training-angle distribution well but fails to transfer to unseen viewpoints.

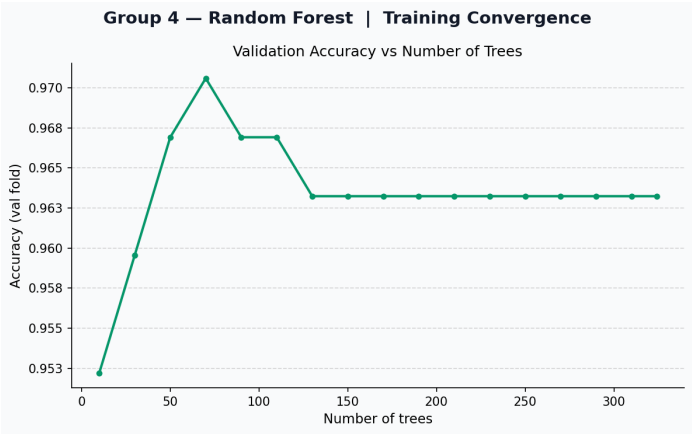

Figure B1: Validation accuracy vs. number of trees for the Random Forest. Convergence is reached around 130 trees; the gap between this in-distribution validation accuracy ($\sim$0.964) and the test accuracy (0.546) reflects the cross-angle covariate shift of Phase 2.

The bootstrap CI on test accuracy (Fig. B2, Table B1) spans $[0.506, 0.587]$, straddling the 0.5 mark. The width of this interval reflects both the difficulty of the task and the relatively small test set, making point estimates alone insufficiently informative.

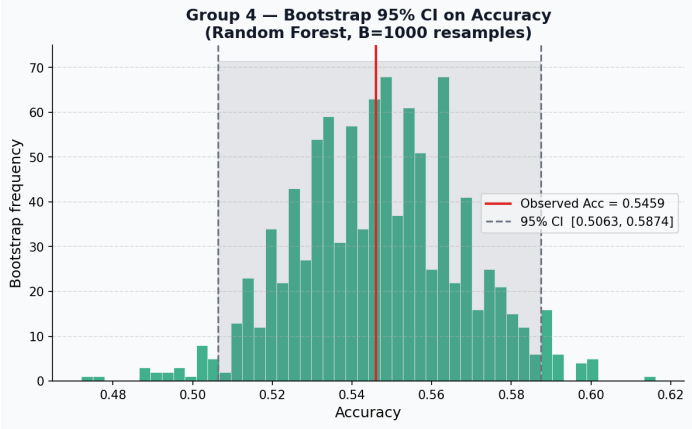

Figure B2: Bootstrap CI on test accuracy ($B = 1000$). The wide interval [0.506, 0.587] reflects genuine uncertainty due to the small test size and covariate shift conditions.

The confusion matrix in Fig. B3 reveals a highly heterogeneous per-class performance. Classes 4 and 5 achieve 0.87 and 1.00 recall respectively, while classes 2, 3, and 10 are classified poorly (0.39, 0.49, and 0.50 recall). Class 5 being perfectly classified while class 2 achieves only 0.39 suggests that the two groups have very different feature distributions in the training data, likely due to species-specific appearance or camera-angle correlation: if class 5 was predominantly captured from one angle in both phases, the model transfers well; classes captured from varied angles in training but from new angles in test will suffer most. The systematic confusion between classes 2 and 4 (0.47 of class 2 predicted as class 4) and between classes 3 and 1/4 further suggests these species share overlapping feature distributions in the 3-band CNN representation, echoing the inter-class similarity we observed in our own pollinator detection task.

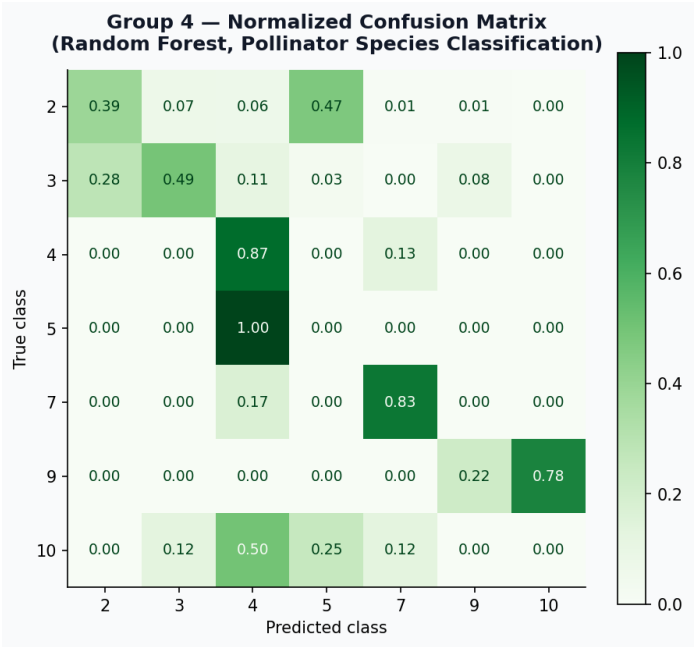

Figure B3: Normalized confusion matrix for the Random Forest on the test set. Per-class recall ranges from 0.39 (class 2) to 1.00 (class 5). Systematic confusions between classes 2, 3, 4, and 10 suggest overlapping feature distributions compounded by cross-angle covariate shift.

Limitations and future directions. The fundamental bottleneck is not the classifier but the covariate shift between training and test viewpoints. Domain adaptation techniques—such as feature alignment or test-time normalization—would be natural next steps. Additionally, having access to the raw images rather than pre-extracted features would allow view-invariant data augmentation (rotations, crops from multiple angles) that could substantially improve cross-angle generalization. Finally, the macro F1 metric, combined with the severe class imbalance, makes per-class calibration (e.g., class-specific thresholds on the predicted probability vector) a promising avenue for closing the gap between per-class and aggregate performance.